GAER: Graph Auto-encoders for Unsupervised Software Architecture Recovery

RAKHSHANDA JABEEN, Electrolux Professional, Sweden
MORGAN ERICSSON, Linnaeus University, Sweden
JONAS NORDQVIST, Linnaeus University, Sweden
ANNA WINGKVIST, Linnaeus University, Sweden

Recovering the modular architecture of software systems from source code remains challenging when documentation is incomplete or outdated. Manual recovery is labor-intensive and does not scale to large systems. Although automated techniques have been proposed, many rely on handcrafted heuristics. They may focus on a limited set of inputs (such as dependencies or text), or combine multiple inputs using fixed rules, rather than integrating structural and semantic cues in a data-driven way.

We introduce GAER, an unsupervised architecture recovery approach based on graph autoencoders. GAER models a system as a heterogeneous dependency graph with typed relations and combines dependency information, folder hierarchy, and code semantics as node features. We study two factors that influence recovery outcomes: the choice of encoder (Graph Attention Network [GAT] vs. Graph Convolution Network [GCN]) and the number of clusters used for the final decomposition. We evaluate GAER across 10 open-source systems and benchmark it against established architecture recovery baselines using standard recovery measures against ground truth mappings. Overall, GAER is competitive, with strong baselines and often achieving higher agreement with ground truth, and the GAT variant generally performs best. By integrating multiple architectural cues in a graph learning framework, GAER produces architectural views that support system comprehension and reduce the effort required to maintain architectural documentation.

CCS Concepts: • **Software and its engineering → Software architectures**; **Software reverse engineering**; • **Computing methodologies → Unsupervised learning**.

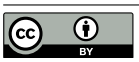
Additional Key Words and Phrases: software architecture recovery, graph neural networks, graph auto-encoders

ACM Reference Format:
Rakhshanda Jabeen, Morgan Ericsson, Jonas Nordqvist, and Anna Wingkvist. 2026. GAER: Graph Auto-encoders for Unsupervised Software Architecture Recovery. *Proc. ACM Softw. Eng.* 3, FSE, Article FSE180 (July 2026), 22 pages. https://doi.org/10.1145/3808187

1 Introduction

Software systems evolve rapidly, making it increasingly difficult to maintain accurate and up-to-date architectural documentation [6]. The original design documents often become outdated or lost as a software project undergoes frequent refactoring, fast development cycles, and codebase changes. This process leads to architectural drift and erosion, where the implemented system diverges from its intended design, complicating maintenance and evolution due to a lack of reliable architectural knowledge [9].

Authors' Contact Information: Rakhshanda Jabeen, Electrolux Professional, Ljungby, Sweden, rakhshanda.jabeen@electroluxprofessional.com; Morgan Ericsson, Linnaeus University, Växjö, Sweden, morgan.ericsson@lnu.se; Jonas Nordqvist, Linnaeus University, Växjö, Sweden, jonas.nordqvist@lnu.se; Anna Wingkvist, Linnaeus University, Växjö, Sweden, anna.wingkvist@lnu.se.



This work is licensed under a Creative Commons Attribution 4.0 International License.
© 2026 Copyright held by the owner/author(s).
ACM 2994-970X/2026/7-ARTFSE180
https://doi.org/10.1145/3808187

Software Architecture Recovery (SAR) addresses this challenge by automatically inferring a system's modular decomposition from its source code. SAR supports program understanding and re-engineering, since manually organizing hundreds or thousands of files into coherent modules is labor-intensive and does not scale. For example, recovering the architecture of a medium-sized system (70K–280K lines of code) can require hundreds of hours of expert effort [11].

Software dependency graphs capture relations among source code entities, and files that implement related functionality often appear close in the graph. Many traditional architecture recovery techniques, therefore, exploit static dependencies and use clustering or search-based optimization to produce decompositions with high cohesion and low coupling [1, 25, 27, 49]. However, shared utilities and layered interactions may introduce many cross-module dependencies, so "connected by an edge" does not necessarily imply "belongs to the same module" [46]. Moreover, dependency relations differ in how strongly they reflect architectural boundaries [24, 47]. The recent dependency-only representation learning method, NEGAR [4], uses node2vec-style embeddings to capture higher-order structural context beyond immediate neighbors, but recovery quality can be sensitive to the choice of dependency types.

Directory structure can reflect how developers organize the codebase into subsystems, while textual cues from identifiers and comments can capture domain concepts and developer intent [8, 11, 13]. Accordingly, some architecture recovery approaches incorporate textual information (e.g., source code and comments) via topic modeling or latent semantic indexing to group artifacts by semantic similarity [8, 13, 43]. These semantic cues can be particularly helpful when structural dependencies are noisy or dominated by cross-cutting utilities, as they provide complementary evidence about intended responsibilities.

More recent techniques combine both structural and non-structural cues, but often integrate them using hand-crafted similarity measures and manually tuned weighting schemes rather than integrating signals in a data-driven way [38, 56]. For example, SARIF [56] augments dependency information with directory structure and textual similarity, but combines these cues using manually designed similarity functions and tuned weights, including dependency-type weights. Since dependency patterns and architectural styles differ across projects, fixed heuristics may not generalize consistently across systems.

We illustrate these challenges using two systems from our study. JabRef contains widely used utility code that creates many cross-module dependencies, making structure-only signals ambiguous; here, directory context and identifier-level semantics can provide complementary evidence for recovering a meaningful decomposition. Chromium, in contrast, is substantially larger and more complex, where fixed fusion rules and manually tuned weights are brittle across diverse dependency patterns; in such settings, learning-based embeddings can better capture global structural context. These observations motivate a representation-learning approach that jointly learns from both structural and non-structural signals, without manually tuning weights across dependency types or information sources.

We propose GAER[1] [18], an unsupervised SAR approach that models the software system as a multi-relational dependency graph and learns relation-aware embeddings of software entities using a heterogeneous Graph Neural Network (GNN) within a Graph Auto-Encoder (GAE) framework. The learned node embeddings are clustered to form the architectural decomposition.

A key design choice in GAER is the encoder. We study two variants, a graph convolution network (GCN) and a graph attention network (GAT), to assess their effectiveness in SAR. We also analyze how varying architectural granularity (i.e., the number of clusters) influences the

[1]The source code and data used are also available at https://github.com/sse-lnu/GAER.git (v.1.1.0).

quality of recovered decompositions. For evaluation, we compare GAER against several established techniques. The primary contributions of this work can be summarized as follows:

(1) **Typed Dependency Graph and Heterogeneous Encoder.** We model each system as a multi-relational dependency graph, where nodes are software entities (files/classes), edges are typed static dependencies, and node features combine semantic signals from code identifiers and directory context from file paths. We use a relation-specific (heterogeneous) GNN encoder with type-specific parameters, so different dependency types are modeled explicitly rather than merged into a single weighted graph.

(2) **Unsupervised Architecture Recovery via GAE.** We train a graph auto-encoder to reconstruct observed dependency edges under a single objective, so the learned embeddings preserve the dependency structure. The resulting node embeddings are then clustered to recover architectural modules.

(3) **Empirical Evaluation.** We evaluate GAER on 10 systems with available ground-truth mappings against established baselines using standard architecture-recovery metrics. We further analyze the impact of encoder choice (GCN vs. GAT), cluster granularity, and run-to-run variability of stochastic methods.

The paper is structured as follows. Section 2 introduces the background concepts underlying our approach. Section 3 describes GAER, including heterogeneous graph construction, the GAE-based learning framework, and the clustering procedure. Section 4 introduces the datasets and baselines, details the experimental setup, and defines the evaluation metrics. Section 5 reports and analyzes the results. Section 6 reviews related work, Section 7 discusses threats to validity, and Section 8 concludes the paper and outlines future directions.

2 Background

Software architecture recovery (SAR) can be viewed as a graph representation learning problem. A software system can be represented as a set of software entities connected by static dependencies such as calls, imports, and inheritance. Together, these dependencies define a multi-relational graph. In addition to the dependency structure, each entity can be associated with attributes such as code identifiers and directory information. The goal is to learn representations that capture architectural similarity, so that entities within the same module are close in the learned vector space.

GNNs are well-suited to SAR because they learn node representations by combining node attributes with graph structure through message passing. At a high level, each node updates its representation by aggregating information from its neighbors along the graph edges [20]. In a software graph, this allows an entity representation to reflect both its own local attributes and the dependency context in which it appears. Moreover, different dependency types can carry different architectural meanings. For example, function-call edges may indicate stronger modular boundaries than type-usage edges [47]. For this reason, relation-specific processing can be used to handle edge types separately rather than merging them into a single graph.

Graph Auto-Encoders (GAEs) [21] provide an unsupervised way to learn such representations. A GAE consists of an encoder, which maps nodes to latent embeddings in a vector space, and a decoder, which attempts to reconstruct observed graph connections from those embeddings. Training encourages the model to produce embeddings that preserve useful structural patterns in the graph while incorporating the input node attributes. After training, these embeddings can be clustered to recover architectural modules.

3 Methodology

Our methodology decomposes software architecture recovery into three stages: (i) constructing a heterogeneous, multi-relational graph from static dependencies, code identifiers, and directory structure; (ii) learning embeddings of software entities with a heterogeneous GNN encoder within a GAE framework; and (iii) clustering the learned embeddings to infer architectural modules. Figure 1 illustrates the end-to-end pipeline of GAER.

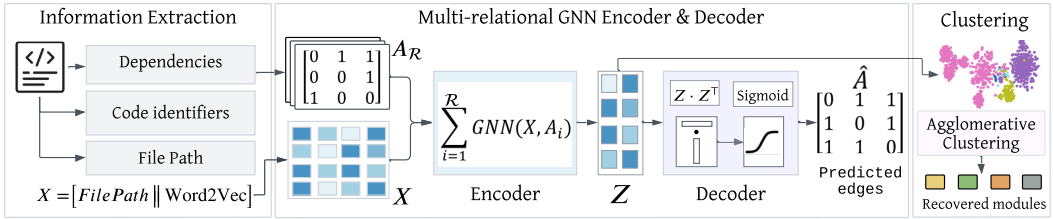


Fig. 1. Overview of the GAER pipeline. Source code is transformed into a multi-relational dependency graph with node features derived from Word2Vec-embedded code identifiers and binary directory-path vectors. A multi-relational heterogeneous GNN encoder learns node embeddings Z within the GAE framework, which are clustered to recover architectural modules.

3.1 Graph Construction

We integrate three sources of information: static dependencies, code identifiers, and directory structure. Dependencies are represented as a directed multi-relational graph, whereas identifiers (class, method, and variable names) and file paths are encoded as node features. Together, these signals provide the structural and contextual information needed for architecture recovery [41, 56].

Formally, we represent the system as a multi-relational directed graph $G = (X, A_{\mathcal{R}})$ with n entities (e.g., files or classes), where $X \in \mathbb{R}^{n \times F}$ is the node feature matrix and $A_{\mathcal{R}} = \{A^{(r)}\}_{r \in \mathcal{R}}$ is the set of relation-specific adjacency matrices. For each relation $r \in \mathcal{R}$, an entry in the adjacency matrix is defined as:

$$A_{ij}^{(r)} = \begin{cases} 1 & \text{if a dependency of type } r \text{ exists from } i \text{ to } j, \\ 0 & \text{otherwise.} \end{cases} \tag{1}$$

We use the static analysis tool *Depends* [36] to extract implemented code entities (e.g., classes and files), their associated program elements (e.g., methods, variables, etc), and typed dependencies from the subject systems. The extracted dependency types (e.g., calls, inheritance, interface implementation, imports, field usage, exceptions) define $\mathcal{R}$ and populate the adjacency matrices $\{A^{(r)}\}_{r \in \mathcal{R}}$.

Node features for each file are derived from textual identifiers and file paths. We normalize code identifiers by splitting *camelCase*, *PascalCase*, and *snake_case*, and remove low-information tokens (stopwords, very short tokens, and tokens appearing in over 90% of files). The remaining tokens are embedded into 100-dimensional vectors using the skip-gram variant of Word2Vec [28], trained for 10 epochs with a context window of 5. The textual representation $\mathcal{T}_i$ of an entity i is then computed by averaging its Word2Vec embeddings and then normalizing them using the ℓ_2 norm.

To encode directory structure, file paths are tokenized and represented as binary bag-of-words vectors. The final feature vector for node i is the concatenation of its binary file-path vector and the Word2Vec embeddings as follows:

$$x_i = \left[\text{Binary(FilePath)}_i \,\middle\|\, \text{Word2Vec}(\mathcal{T}_i)\right] \tag{2}$$

3.2 GAER Model

Given the heterogeneous graph constructed in Section 3.1, GAER learns node embeddings by training a graph auto-encoder to reconstruct observed dependencies. The encoder maps the attributed multi-relational graph to an embedding matrix Z, where each row of Z corresponds to the learned embedding of a software entity:

$$Z = f_\theta(X, A_\mathcal{R}) \tag{3}$$

The decoder then uses these embeddings to estimate which pairs of entities should be connected.

3.2.1 Relation-Specific GNN Encoder. We instantiate the encoder with a relation-specific GNN operating on the software graph $G = (X, A_\mathcal{R})$, where X encodes node features (code identifiers and file paths) and $A_\mathcal{R}$ encodes typed dependencies. We use a single encoder layer, so each node updates its representation using messages from its one-hop neighbors. This produces embeddings that reflect both node attributes and local dependency structure.

We consider two encoder variants, GCN [20] and GAT [52], to compare fixed degree-normalized aggregation with learnable neighbor weighting. Because different dependency types can reflect architectural boundaries with different strengths [4, 47], we use a heterogeneous encoder with type-specific parameters, allowing each relation type to be handled separately. Such heterogeneous encoders have also been shown to outperform homogeneous counterparts in related tasks [17].

Intuitively, each relation $r \in \mathcal{R}$ is processed by its own message-passing channel with separate learnable parameters, and the relation-specific outputs are summed to obtain node embeddings. Formally, the encoder produces an embedding matrix $Z \in \mathbb{R}^{n \times F'}$. Let $\mathbf{h}_i^{(l)}$ denote the embedding of node i at layer l, $\mathcal{N}_i^r$ denote the neighbors of node i under relation r, and let d_i^r denote the degree of node i under relation r. In our setup, we use a single encoder layer, i.e., $\mathbf{h}_i^{(0)} = x_i$ and $z_i = \mathbf{h}_i^{(1)}$, where z_i denotes the final F'-dimensional embedding of node i. For each dependency type r, we apply a learnable relation-specific linear transformation $W^r \in \mathbb{R}^{F' \times F}$ before aggregating neighbor messages.

Heterogeneous GCN. In the heterogeneous GCN variant, node i is updated by averaging degree-normalized messages from its neighbors for each dependency type. Messages from relation-specific neighbors $j \in \mathcal{N}_i^r$ are first linearly transformed using W^r and normalized by the degrees d_i^r and d_j^r. The aggregated message is then passed through a non-linear activation function (ReLU) to obtain the updated node representation:

$$\mathbf{h}_i^{(1)} = \mathrm{ReLU}\left(\sum_{r \in \mathcal{R}} \sum_{j \in \mathcal{N}_i^r} \frac{1}{\sqrt{d_i^r d_j^r}} W^r \mathbf{h}_j^{(0)} \right) \tag{4}$$

Degree normalization controls the influence of high-degree nodes and stabilizes aggregation across different dependency types.

Heterogeneous GAT. In the heterogeneous GAT variant, degree-based normalization is replaced by learnable attention weights inspired by the self-attention mechanism [51]. The attention weights quantify the importance of each neighbor $j \in \mathcal{N}_i^r$ when updating node i under dependency type r. For each relation r, node features are first linearly transformed using W^r, and attention scores e_{ij}^r are computed and normalized with a softmax function to obtain α_{ij}^r. The attention-weighted messages are then aggregated and passed through a non-linear activation function (ELU) to produce

the updated node representation:

$$\mathbf{h}_i^{(1)} = \mathrm{ELU}\left(\sum_{r\in\mathcal{R}}\sum_{j\in\mathcal{N}_i^r} \alpha_{ij}^r \, W^r \mathbf{h}_j^{(0)}\right) \tag{5}$$

3.2.2 ***Decoder and Reconstruction Loss.*** The decoder g_ϕ produces an estimated adjacency matrix $\hat{A} = g_\phi(Z)$, where each entry $\hat{A}_{ij}$ represents the predicted probability of an edge between the node pair (i, j). It is computed by taking the inner product of the corresponding node embeddings and applying a logistic sigmoid function as shown in Eq. 6, where $(\cdot)^\top$ denotes transposition.

$$\hat{A}_{ij} = \frac{1}{1 + \exp\left(-z_i^\top z_j\right)} \tag{6}$$

For each dependency type $r \in \mathcal{R}$, let $E_r = \{(i, j) \mid A_{ij}^{(r)} = 1\}$ denote the observed edge set. Since the decoder reconstructs a single symmetric adjacency matrix, we treat dependencies as undirected for reconstruction by ignoring edge direction. The positive edge set is therefore defined as the union of observed edges across all relation types, $E^+ = \bigcup_{r\in\mathcal{R}} E_r$.

Negative edges are node pairs $(i, j) \notin E^+$. Because the number of such pairs is large, we sample a set of negative pairs S^- uniformly at random from the complement of E^+, matching the number of sampled negatives to the number of positive edges. The reconstruction loss then encourages high scores for observed edges and low scores for sampled negative edges using the following binary cross-entropy objective:

$$\mathcal{L} = -\sum_{(i,j)\in E^+} \log \hat{A}_{ij} - \sum_{(i,j)\in S^-} \log\left(1 - \hat{A}_{ij}\right) \tag{7}$$

By learning embeddings that reconstruct observed dependency patterns, the GAE captures structural regularities in the software graph while also exploiting the node attributes provided as input. These embeddings are then used for downstream software architecture recovery.

3.3 Clustering for Architecture Recovery

To recover architectural modules from node embeddings learned from the GAE, we apply Agglomerative Hierarchical Clustering (AHC) [33] with Ward linkage. AHC starts with each entity as its own cluster and iteratively merges the closest clusters. Rather than analyzing the full hierarchy, we output a flat partition by selecting the number of clusters k.

GAER supports either a user-defined k or automatic selection, where k is chosen from a candidate search range $\mathcal{K}$ using the elbow criterion. For each $k \in \mathcal{K}$, we compute a Within-Cluster Sum of Squares (WCSS) value from the resulting AHC partition, defined as the sum of squared Euclidean distances from each entity to the mean (centroid) of its assigned cluster. Since WCSS decreases monotonically as k increases, we select k using a standard distance-to-chord heuristic on the WCSS curve, i.e., the k that maximizes the perpendicular distance to the line connecting the smallest and largest candidate k. For scalability, on large systems, the WCSS curve is estimated on a random subsample of 2 000 entities, and the final AHC clustering is then applied to the full dataset using the selected k. Algorithm 1 summarizes the GAER algorithm.

4 Experiments

We evaluate GAER and a set of representative baseline methods on a collection of open-source software systems with known ground truth (GT) labels. This section describes the subject systems, the baseline techniques and their configuration, the GAER experimental setup, and the evaluation

Algorithm 1: GAER

Input: Software system with n entities, their file paths, identifier tokens, and typed dependencies $\{E_r\}_{r\in\mathcal{R}}$;
Embedding dimension d; training epochs ϵ; optimizer;
Clustering: user-defined k or candidate range $\mathcal{K}$
Output: Recovered architecture (module assignment) $\mathcal{A}$

Step 1: Construct Multi-relational Adjacency Matrices and Node Features
$X \leftarrow \left[\textsc{Binary}(\text{file paths}) \| \textsc{Word2Vec}(\text{code})\right]$; // node features $X \in \mathbb{R}^{n\times F}$
Construct $A_{\mathcal{R}} = \{A^{(r)}\}_{r\in\mathcal{R}}$ from $\{E_r\}_{r\in\mathcal{R}}$; // binary adjacency matrices

Step 2: Representation Learning (GAE)
Initialize encoder f_θ (het. GCN/GAT) and decoder g_ϕ (inner-product);
$E^+ \leftarrow \bigcup_{r\in\mathcal{R}} E_r$; // observed dependencies
for $t = 1$ **to** ϵ **do**
  Sample S^- from the complement of E^+;
  $Z \leftarrow f_\theta(X, A_{\mathcal{R}})$; // node embeddings, $Z \in \mathbb{R}^{n\times F'}$
  $\hat{A} \leftarrow g_\phi(Z)$; // edge probabilities, $\hat{A} \in \mathbb{R}^{n\times n}$
  Compute loss $\mathcal{L}(E^+, S^-)$;
  Update f_θ, g_ϕ using optimizer to minimize $\mathcal{L}$;

Step 3: Clustering if k *is user-defined* **then**
  $k^* \leftarrow k$;
else
  $k^* \leftarrow \textsc{Elbow}\Big(\text{WCSS}(\text{AHC}(Z,k))_{k\in\mathcal{K}}\Big)$;
$\mathcal{A} \leftarrow \text{AHC}(Z, k^*)$; **return** $\mathcal{A}$

procedures and metrics used to benchmark performance. Our experimental study is structured to address the following research questions:

- **RQ1:** How well does GAER perform in terms of established architecture recovery measures compared to existing baselines?
- **RQ2:** What is the impact of the number of clusters (granularity) on the recovered decomposition and recovery metrics?
- **RQ3:** How stable are stochastic architecture recovery techniques across repeated runs in terms of evaluation metric variability?

4.1 Subject Systems

We evaluate our approach on ten open-source systems that have been widely studied in prior work on SAR and Code-to-Architecture (C2A) mapping [4, 37, 41, 56]. The systems cover different domains and vary in size. Using published GT architectures is common in SAR as producing reliable C2A mappings requires substantial manual effort and expert knowledge [11]. These established GT architectures also allow for direct comparability with previous studies. Table 1 summarizes the key characteristics of the systems, including the number of architectural components k.

For ArchStudio (AS4), Hadoop, OODT, and Bash, we use the C2A mapping curated by Garcia et al. [11]. These mappings were produced using a certification process in which researchers first recovered an initial architectural decomposition from documentation and source code, and then refined it through iterative validation with system experts. The reported average number of person-hours to obtain the mapping was approximately 107 per system for these medium-sized systems. The same line of work also provides the GT mapping for Chromium ($\approx$ 11M LoC), which required long-term collaboration with developers over roughly two years [23].

JabRef and TeamMates (T.Mates) were studied in the Software Architecture Erosion and Architectural Consistency (SAEroCon) workshops in 2016 and 2017, where the system experts provided

Table 1. Properties and versions of software systems.

Name	Version	Language	LoC	|Files|	|Dependencies|	k	Domain
ArchStudio	4	Java	280K	583	9.1K	57	Architecture Modeling
Hadoop	0.19.0	Java	200K	591	24.7K	67	Distributed Processing
JabRef	v3.7	Java	59K	1 180	26.3K	6	Reference Management
OODT	0.2	Java	180K	876	18K	216	Data Management
TeamMates	v5.11	Java	55K	778	49.7K	15	Peer Feedback System
Chromium	svn-171054	C++	11M	18 343	713.5K	67	Web Browser
Bash	4.2	C	115K	292	1.8K	14	Shell
HDC	46ff87	C++	26K	207	2.5K	11	Camera Interface
HDF	0e196f	C	153K	153	2.8K	9	Driver Subsystem
Libxml	2.4.22	C	81K	82	9.3K	17	XML Parser

the reference architecture and the corresponding C2A mapping [37]. The GT C2A mappings for HDF, HDC, and Libxml were developed in collaboration with industry partners and are provided in the SARIF benchmark suite [56], with labels distributed via its GitHub[2] repository.

4.2 Baseline Methods

We compare GAER against established SAR techniques that use different information sources and recovery strategies. We include dependency-driven methods (ACDC [49], WCA [27], LIMBO [1], FCA [48], NEGAR [4]), a semantic-similarity method (ARC [13]), and hybrid methods that combine structural and semantic cues (SADE [38], SARIF [56]). A broader overview of these techniques is provided in the Table 8. We use SARIF and NEGAR as primary reference methods in our analysis.

SARIF [56] integrates three information sources: dependency graph, textual features extracted from code and comments, and directory structure into a unified weighted graph. Dependency types are assigned weights tuned to maximize modularity, while additional edge weights are derived from TF-IDF similarity and topic modeling of code identifiers and comments, as well as from directory co-location. The resulting weighted graph is clustered using the Louvain community detection algorithm [5], with the resolution parameter γ controlling the clustering granularity.

NEGAR [4] learns node embeddings from an unweighted dependency graph. To do so, it applies node2vec [14] and generates a fixed number of second-order random walks for each node i. The return parameter p and in-out parameter q control the balance between local and outward exploration. Each walk yields a node sequence, and the collection of sequences is treated as a corpus. A word2vec-style SkipGram [28] model is then trained to learn embeddings by predicting the neighborhood of node i within a context window along these sequences. Finally, k-means clustering is applied to the embeddings to recover architectural modules.

4.3 Experimental Configuration

In preliminary experiments on a subset of systems, we explored different encoder depths (1–2 layers), embedding dimensions (64, 128, and 256), and training lengths (30, 50, and 100 epochs). Increasing the number of epochs yielded only minor differences in recovery metrics, so we use 30 epochs and a consistent GAER configuration across all datasets. Specifically, we use a single-layer GCN or GAT encoder with 128-dimensional embeddings (a single attention head for GAT), trained with Adam at a learning rate of 1e-4. For very small systems (<100 entities), we use a learning rate of 1e-3 and train for 50 epochs; this applies to Libxml.

[2] https://github.com/anonymous2f4a9d/SARIF_FSE23

We reuse baseline results (ACDC, ARC, WCA, LIMBO, FCA, and SADE) from the SARIF GitHub[2]. Methods that require a user-specified number of clusters (WCA, LIMBO, ARC) are evaluated for $k \in [10, 100]$ with a step of 5; ARC additionally varies the number of topics from 50 to 150 with a step of 10. We report the best result observed for each metric across the evaluated settings. We run SARIF using the same replication package across all datasets, with the default resolution parameter ($\gamma = 1.7$) reported by the authors. We observed no run-to-run variation across repeated executions, so we report a single run per dataset.

We apply NEGAR following Chen et al.'s recommended configuration and tuning setup (Table II in [4]). We tune on AS4/Bash/Hadoop and use the reported Chromium setting; for the remaining datasets, we keep a consistent configuration (128 dimensions, walk length 30, context size 15, 200 walks per node, in-out and return parameters $p = q = 1$, 5 negative samples, and 5 training epochs). While NEGAR evaluates dependency types via separate pipelines, we merge all dependency types into a single dependency graph, achieving comparable performance. To support our unknown-k evaluation, we additionally select k from the same user-defined search range described in Sec. 3.3 using k-means on the learned embeddings. We use the same dependencies extracted by Depends across GAER, NEGAR, and SARIF.

All experiments are conducted on a server with 2x AMD EPYC 7413 CPUs (24 cores, 2.65GHz base clock), 512GB of RAM, and an NVIDIA A100 GPU (SMX4, 40GB RAM).

4.4 Performance Evaluation

Multiple reasonable architectural decompositions may exist for the same software system, since experts can disagree on architectural boundaries and design intent [11]. Consequently, a recovered decomposition may differ from the particular GT architecture used for evaluation. To avoid reliance on any single criterion, we report a set of complementary metrics using standard definitions and computation procedures, as summarized in prior architecture-recovery evaluations [24]. Specifically, we report MoJoFM, A2A, and $C2C_{cvg}$ as GT-alignment accuracy measures commonly used in architecture recovery, and TurboMQ as a GT-independent modularization-quality metric. In addition, we report the Adjusted Rand Index (ARI) [16], a widely used clustering-agreement measure that ranges from −1 to 1 (0 indicates chance-level agreement and 1 a perfect match). We implemented the metrics in Python based on descriptions and previous implementations.

MoJoFM [53] is defined by the formula,

$$\mathrm{MoJoFM}(A_1, A_2) = \left(1 - \frac{mno(A_1, A_2)}{\max(mno(\forall A_1, A_2))}\right) \times 100 \tag{8}$$

where $mno(A_1, A_2)$ is the minimum number of *move* and *join* operations required to transform the recovered architecture A_1 into the GT architecture A_2 at the cluster-level, and the denominator is the maximum possible number of such operations used for normalization. MoJoFM is reported to be more accurate than other measures; however, it is less sensitive to over-fragmentation because of its relatively cheap *join* operations. Thus, many small clusters can still yield a high score when they largely align with GT modules [24].

Architecture-to-Architecture (A2A) [22] extends MoJoFM by incorporating operations for adding, removing, and moving entities and clusters. It is defined as follows:

$$\mathrm{A2A}(A_1, A_2) = \left(1 - \frac{mto(A_1, A_2)}{aco(A_1) + aco(A_2)}\right) \times 100 \tag{9}$$

where mto is the number of edits required to convert A_1 into A_2, and aco is the cost of constructing the respective architecture from scratch. One limitation of A2A is that it can assign high similarity

scores even when two architectures are structurally quite different, thereby overestimating their true similarity [23].

Cluster-to-Cluster Coverage $C2C_{cvg}$) [12] measures the percentage of GT clusters that are sufficiently matched by clusters in the recovered architecture. It is defined as follows:

$$C2C_{cvg}(A_1, A_2) = \frac{|\text{simC}(A_1, A_2)|}{|A_2|} \times 100 \tag{10}$$

where, $\text{simC}(A_1, A_2)$ denotes the number of clusters in A_2 that overlap with some cluster in A_1 beyond a user-defined coverage threshold th_{cvg}. By convention, $th_{cvg} = 0.10,\ 0.33$, and 0.50 correspond to weak, moderate, and majority matches.

Normalized TurboMQ [24] is independent of the GT architecture and assesses modular quality based on the dependency graph by rewarding intra-cluster cohesion and penalizing inter-cluster coupling. Let k be the number of recovered clusters. For each cluster i, let μ_i denote the total weight of intra-cluster dependencies, and let ε_{ij} denote the total weight of dependencies from cluster i to cluster j. TurboMQ is defined as:

$$\text{TurboMQ} = \sum_{i=1}^{k} \frac{\mu_i}{\mu_i + 0.5 \times \sum_{j \neq i} (\varepsilon_{ij} + \varepsilon_{ji})} \tag{11}$$

TurboMQ can favor decompositions with many clusters [24]. To reduce this bias, we report only the normalized TurboMQ score (TurboMQ/k).

5 Results and Analysis

This section presents the empirical evaluation of GAER with respect to the three research questions and includes additional analyses of scalability. We first compare GAER to established baselines using standard architecture recovery metrics (RQ1), then analyze the impact of architectural granularity (RQ2), and finally examine the stability of the stochastic methods across repeated runs (RQ3). Since GAER and NEGAR involve stochastic components (e.g., random initialization and negative sampling), we repeat each configuration 100 times with different random seeds to explicitly quantify run-to-run variability and assess stability. Unless stated otherwise, we report the median performance across runs.

5.1 Evaluation and Comparison of Recovered Architectures (RQ1)

To address RQ1, we evaluate GAER by comparing recovered architectures against the GT using MoJoFM, A2A, $C2C_{cvg}$, and ARI. We compare the GAER variants (GCN and GAT) with ACDC, LIMBO, FCA, SADE, SARIF, two WCA variants, and NEGAR. Table 2 reports results in the unknown-k setting, where k is selected from the practical range 10–30 using the fixed selection criterion described in Sec. 3.3 (applied to GAER and NEGAR).

The metrics in Table 2 capture different aspects of alignment with the GT mapping. Overall, GAER achieves the strongest results across most datasets and metrics, with GAT often best and GCN close behind. OODT is the main exception because its reference decomposition has more than 200 modules, which lie far outside our $\mathcal{K}$ range (10-30); ACDC, ARC, and FCA achieve the highest scores there at much finer granularity. While we report all baselines for completeness, several are competitive only on a subset of systems or metrics; we therefore focus the discussion on the most informative and comparable reference techniques, SARIF and NEGAR, and highlight cases where other approaches are most competitive.

MoJoFM shows a clear trend: GAER often achieves higher GT alignment than the baselines, with GAT usually best and GCN close behind. Compared with SARIF, GAER improves MoJoFM on several datasets (e.g., Chromium, Hadoop, HDF, and T.Mates). NEGAR is also a strong structure-only

Table 2. Similarities between recovered and GT architecture. The metrics MoJoFM (M), A2A (A), $C2C_{cvg}$ (C), and ARI (I) are scaled by 100. The top 2 values per metric are highlighted.

Tech.	ArchStudio				Bash				Chromium				Hadoop				OODT				HDC				HDF				Libxml				JabRef				T.Mates			
	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I
ACDC	73	86	37	20	50	81	10	13	48	82	1	7	46	82	0	13	44	85	39	45	49	82	0	24	38	79	4	13	34	83	25	13	–	–	–	–	–	–	–	–
ARC	57	86	20	32	28	80	5	3	33	83	0	2	47	83	43	24	30	82	50	12	37	81	4	8	36	80	0	12	29	83	26	6	–	–	–	–	–	–	–	–
FCA	60	82	19	10	41	80	7	13	58	76	0	3	49	82	19	11	50	86	42	23	53	78	2	10	32	78	3	15	37	84	24	8	–	–	–	–	–	–	–	–
LIMBO	27	80	2	0	27	80	2	0	31	78	0	0	17	80	1	2	12	77	0	0	25	80	5	1	20	81	0	-53	22	84	24	2	–	–	–	–	–	–	–	–
WCA_{UE}	31	84	0	3	40	81	2	8	31	82	0	2	16	79	13	0	12	78	32	0	51	80	2	11	45	82	7	26	32	84	15	9	–	–	–	–	–	–	–	–
WCA_{UENM}	31	84	0	3	39	80	2	7	58	82	0	2	16	79	13	0	12	78	32	0	48	80	1	10	44	82	5	23	27	83	14	6	–	–	–	–	–	–	–	–
SADE	42	83	29	11	45	86	17	21	58	87	35	26	41	83	44	23	21	78	43	13	72	88	17	41	49	88	33	30	34	84	17	22	–	–	–	–	–	–	–	–
SARIF	66	87	56	35	76	84	13	39	60	86	23	27	47	84	13	29	30	80	41	20	89	89	33	53	62	87	27	37	47	88	45	26	89	83	7	17	76	87	33	39
NEGAR	63	89	45	53	65	87	22	33	64	88	48	39	51	84	27	27	19	76	13	12	71	87	17	33	57	86	16	31	41	86	29	18	77	82	0	9	67	85	7	25
GCN	66	86	41	40	90	92	53	53	73	91	58	59	57	86	33	40	24	77	13	17	89	88	28	45	88	92	44	72	66	92	55	44	90	85	9	21	87	89	29	48
GAT	70	87	53	39	91	92	63	57	76	91	61	60	24	77	16	17	89	87	26	42	92	92	50	70	66	91	53	41	92	85	11	22	95	90	44	46				

baseline and often reaches MoJoFM values around the 60s; it can exceed SARIF on some systems, including Hadoop and Chromium. Libxml is a notable exception for NEGAR: its dependency graph is small and sparse, so random-walk embeddings have limited structural context to learn from.

A2A remains high across most datasets and methods, which limits its ability to clearly separate methods. Still, the overall ranking is broadly consistent with MoJoFM, with GAER usually strongest for both encoders. NEGAR is also competitive and even surpasses SARIF on A2A for Bash, AS4, and Chromium (e.g., Bash: 87 vs. 84).

$C2C_{cvg}$ is more sensitive to splitting GT modules than MoJoFM and A2A. It checks whether each GT module is covered by at least 50% by a single recovered cluster (with $th_{cvg} = 0.5$), so it can drop sharply when GT modules are split across multiple clusters. For example, JabRef has only six GT modules, and searching k in 10–30 can produce finer-grained clusters that split these modules, leading to low $C2C_{cvg}$ values. Overall, GAER achieves strong coverage on Bash, Chromium, Hadoop, HDF, Libxml, and T.Mates, indicating that many GT modules are captured by a dominant recovered cluster. $C2C_{cvg}$ also highlights cases where GAER does not lead; SARIF attains higher values on ArchStudio and HDC.

ARI is also sensitive to fragmentation, but it is more stable than $C2C_{cvg}$ because it does not use a fixed coverage threshold. GAER achieves strong ARI on Bash, Chromium, Hadoop, HDF, Libxml, and T.Mates. AS4 is a major exception, in which NEGAR achieves the highest ARI, suggesting that AS4 aligns particularly well with structural signals. Another exception is HDC, where SARIF has a higher ARI. These cases highlight that different metrics can favor distinct decompositions, motivating our subsequent analyses of the number of clusters (RQ2) and stability (RQ3).

Overall, the results show two consistent patterns. First, using information beyond the dependency graph can help recover decompositions that align better with the intended architecture. Across several datasets, GAER and SARIF often outperform structure-only baselines such as NEGAR, suggesting that directory and semantic cues provide a useful signal when dependencies are noisy or incomplete. Second, Chromium highlights a different limitation. On this large system with diverse dependency patterns, the representation-learning methods (GAER and NEGAR) outperform SARIF and all other techniques across all metrics, indicating that fixed fusion rules and manually tuned weights do not generalize well at scale. Together, these findings motivate learning-based integration of multiple cues and show that no single information source or fusion strategy is best for every system.

Statistical Significance. We conduct statistical significance tests to verify that the observed improvements are systematic rather than due to stochasticity in training. Since GAT is the stronger variant of GAER, we focus on pairwise comparisons between GAT, NEGAR, and SARIF. Since metric

values from repeated runs are not normally distributed, we use nonparametric tests. We apply the one-sample Wilcoxon signed-rank test [54] when comparing the 100-run distributions of GAER and NEGAR against SARIF's single score, and the Mann–Whitney U test [26] when comparing independent 100-run distributions between the stochastic methods.

We also report Cliff's delta (δ)[7] to quantify effect size. δ ranges from −1 to 1, and its sign indicates which method tends to achieve higher scores, with positive values favoring the first method and negative values favoring the second. We interpret the effect size using standard thresholds: negligible ($|\delta| < 0.147$), small ($|\delta| < 0.33$), medium ($|\delta| < 0.474$), and large ($|\delta| \geq 0.474$). Table 3 summarizes these pairwise comparisons (median differences, significance, and effect sizes) across the datasets and metrics included in this analysis.

Table 3. Median differences of methods for all metrics across datasets. Positive values indicate that the first method outperforms the second. Effect size magnitude based on $|\delta|$: N (negligible), S (small), M (medium), L (large) are shown in parentheses. Entries marked with † are not statistically significant ($p \geq 0.05$), and highlighted cells indicate cases where the second method performs significantly better.

	GAT − GCN				GAT − SARIF				GAT − NEGAR				SARIF − NEGAR			
Dataset	MoJoFM	A2A	C2C$_{cvg}$	ARI	MoJoFM	A2A	C2C$_{cvg}$	ARI	MoJoFM	A2A	C2C$_{cvg}$	ARI	MoJoFM	A2A	C2C$_{cvg}$	ARI
AS4	+3.19(L)	+0.5(M)	+11.5(L)	+0.2(N)†	+3.7(L)	−0.4(L)	−3.4(S)	+4.8(L)	+6.6(L)	−0.9(M)	+2.6(S)†	−13.0(L)	+2.9(L)	−0.5(N)	+6.0(L)	−17.9(L)
Bash	+1.1(L)	+0.3(M)	+9.6(L)	+3.1(L)	+15.1(L)	+8.1(L)	+49.5(L)	+17.5(L)	+26.6(L)	+4.6(L)	+40.3(L)	+23.8(L)	+11.5(L)	−3.5(L)	−9.2(L)	+6.3(L)
Chrome	+3.1(L)	+0.5(M)†	+2.7(S)	+1.0(N)†	+16.1(L)	+5.4(L)	+37.6(L)	+32.8(L)	+12.6(L)	+3.6(L)	+12.5(L)	+20.7(L)	−3.4(M)	−1.8(L)	−25.0(L)	−12.1(L)
Hadoop	+8.6(L)	+1.9(L)	+25.0(L)	+12.4(L)	+19.0(L)	+3.7(L)	+45.4(L)	+23.4(L)	+14.6(L)	+3.9(L)	+31.9(L)	+25.9(L)	−4.4(L)	+0.1(M)	−13.5(L)	+2.6(L)
HDC	+0.3(S)†	−0.8(M)	−1.5(S)	−2.8(L)	+0.39(S)	−2.1(L)	−6.7(L)	−11.6(L)	+18.4(L)	−0.3(N)†	+9.6(L)	+9.2(L)	+18.0(L)	+1.9(S)	+16.3(L)	+19.8(L)
HDF	+4.9(L)	−0.7(M)	+6.3(L)	−2.3(S)	+30.3(L)	+5.2(L)	+23.0(L)	+32.6(L)	+35.4(L)	+6.1(L)	+34.2(L)	+38.2(L)	+5.1(L)	+0.8(M)	+11.2(L)	+5.7(L)
Libxml	+0.0(N)†	−0.3(S)	−2.4(S)	−3.1(S)	+18.8(L)	+3.5(L)	+7.6(L)	+15.3(L)	+24.7(L)	+5.2(L)	+24.1(L)	+23.6(L)	+5.9(L)	+1.7(L)	+16.4(L)	+8.3(L)
JabRef	+3.2(L)	+0.17(N)†	+2.4(S)†	+0.4(N)	+3.9(L)	+1.9(L)	+4.4(L)	+4.9(L)	+16.3(L)	+3.0(L)	+11.41(L)	+13.3(L)	+12.3(L)	+1.0(L)	+7.0(L)	+8.4(L)
T.Mates	+7.1(L)	+0.5(S)	+15.2(L)	−1.7(M)	+18.5(L)	+2.7(L)	+10.8(L)	+6.8(L)	+27.1(L)	+4.5(L)	+36.3(L)	+21.0(L)	+8.6(L)	+1.7(M)	+25.6(L)	+14.2(L)
Average	+3.50	+0.23	+7.64	+0.80	+13.98	+3.11	+18.69	+14.06	+20.26	+3.30	+22.55	+18.08	+6.28	+0.16	+3.87	+3.92

Table 3 shows that many differences are systematic; several comparisons are statistically significant and often have medium-to-large effect sizes. Since we select k from a range (10–30), both GAER encoders produce decompositions at a manageable granularity, rather than extremely fine-grained clusterings. Overall, the gap between the two GAER encoders is modest. On average, GAT improves over GCN by +3.5 MoJoFM and +7.64 C2C$_{cvg}$, while differences on A2A (+0.23) and ARI (+0.80) are small. ARI is mixed, favoring GCN on HDC, HDF, Libxml, and T.Mates, which is consistent with GCN sometimes producing more compact partitions. In contrast, GAT more often improves MoJoFM and C2C$_{cvg}$.

Compared with SARIF, GAER-GAT yields positive results across most datasets and metrics, often with large effects. The clearest exceptions are AS4 and HDC. On AS4, SARIF is higher on A2A and C2C$_{cvg}$, while on HDC, it is higher on A2A, C2C$_{cvg}$, and ARI. Overall, the aggregate row shows that GAT's gains over SARIF are largest on MoJoFM, C2C$_{cvg}$, and ARI, while A2A differences are smaller.

The GAT vs. NEGAR comparison shows a consistent advantage for GAER across most datasets and metrics, with only a few exceptions (e.g., AS4 on ARI). In contrast, SARIF vs. NEGAR is mixed: SARIF has a modest overall advantage on MoJoFM, C2C$_{cvg}$, and ARI, but not on A2A, and it trails NEGAR on systems such as Chromium and Hadoop. Overall, the results support two conclusions: both GAER variants are competitive, and GAER-GAT is the stronger default setting.

Scalability. To assess scalability, we report end-to-end runtime for GAER variants and compare it with NEGAR and SARIF (Table 4). We report source code processing separately, since extracting text and dependencies (via ctags [50] and Depends) is performed once per system and reused across methods. For GAER, total runtime includes graph construction, Word2Vec embedding generation,

graph auto-encoder training, agglomerative hierarchical clustering, and selecting the best k within the 10–30 search range. We also include metric computation time. The runtime measurements confirm that our approach is practical and scalable for large-scale architecture recovery tasks.

Table 4. Total time consumption (in seconds) for the GAER (GCN and GAT), NEGAR, and SARIF pipelines. Data extraction is the time required to extract text and dependencies from the source code (using ctags + Depends). We used the default configuration for GAER and SARIF. For NEGAR, we found that 16 parallel tasks provided the best balance across the datasets.

Technique	ArchStudio	Bash	Chromium	Hadoop	OODT	HDC	HDF	Libxml	JabRef	T.Mates
SARIF	48.43	50.05	1 976.79	70.06	53.80	13.19	25.24	36.48	77.36	56.35
NEGAR	4.26	19.12	289.48	5.14	16.89	15.42	7.69	0.41	8.95	8.19
GAER-GCN	1.83	0.57	36.78	1.62	18.84	0.61	0.49	0.61	1.96	1.62
GAER-GAT	2.09	0.74	37.29	1.90	13.08	0.84	0.67	0.81	2.25	1.91
Data extraction	53.12	11.08	2 796.84	58.00	21.36	12.35	36.49	10.81	27.01	30.65

Answer to RQ1: Across most systems, GAER achieves higher (or comparable) alignment with the GT architecture than the baselines on standard recovery metrics. GAT is often best, with GCN usually close behind. The clearest gains appear in MoJoFM and ARI. Improvements in $C2C_{cvg}$ are also common, but this metric is sensitive to how modules are split or merged. In contrast, A2A remains high across most methods, thereby separating methods less clearly. Overall, approaches that integrate multiple information sources or use representation learning tend to be the most competitive, although the best choice can vary by system.

5.2 Impact of Architectural Granularity (RQ2)

Architectural granularity is a design choice rather than a single objectively correct answer, and systems are often organized at different levels of modular abstraction. Our approach recovers architectures by clustering node embeddings into a flat k-partition, so k directly determines the resolution of the recovered decomposition and can substantially influence the GT-alignment metrics.

Several techniques in our comparison also decide the number of clusters on their own and return a partition without a user-chosen k. While this avoids fixing a single resolution in advance, the number of clusters produced by the method can vary widely across systems (and, for stochastic methods, across runs). As shown in Table 6, some approaches yield very fine-grained decompositions of large systems, such as Chromium, producing hundreds or thousands of clusters, which are difficult to interpret as an architectural overview. We therefore evaluate all recovered decompositions as flat partitions and treat granularity as a controlled modeling choice, rather than cutting the hierarchical dendrogram at a fixed distance threshold.

To answer RQ2, we evaluate GT alignment under two granularity views as shown in Table 5. First, we vary the cluster search ranges $\mathcal{K}$ (10–20, 10–30, 10–50, and 10–100) for GAER-GAT to study how widening the search interval shifts the recovered architecture toward coarser or finer decompositions. Second, we report metrics for both GAER variants and NEGAR when $k = k_{GT}$ to assess their potential for alignment with GT at the reference resolution.

Across datasets, widening $\mathcal{K}$ primarily affects performance due to granularity mismatch. When the range is too small for a system with many GT modules, the recovered architecture becomes too coarse, resulting in multiple GT modules being merged. When the range is too large for a system with few GT modules, the recovered architecture tends to be overly fine-grained, with GT modules split across multiple clusters. MoJoFM can stay high even under such splits, while ARI and $C2C_{cvg}$ typically decrease more clearly when fragmentation increases.

Table 5. Performance under varying architectural granularity. The top block shows GAER-GAT results evaluated with k selected from progressively increasing search ranges $\mathcal{K}$. The bottom block reports results for $k = k_{\mathrm{GT}}$ in GAER (GAT, GCN) and NEGAR. Each entry is the median over 100 runs; MoJoFM (M), A2A (A), $\mathrm{C2C}_{\mathrm{cvg}}$ (C), and ARI (I) are scaled by 100. The top scores per dataset are highlighted.

$\mathcal{K}$	ArchStudio				Bash				Chromium				Hadoop				OODT				HDC				HDF				Libxml				JabRef				T.Mates			
	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I
10–20	67	86	55	43	91	93	65	58	72	91	64	61	62	87	57	49	20	76	12	14	91	90	43	55	94	95	57	79	62	91	53	41	92	86	14	24	93	90	47	51
10–30	70	87	53	40	91	92	63	57	76	91	61	60	66	88	59	52	24	77	16	17	89	87	26	42	92	92	50	70	66	91	53	41	92	85	11	22	95	90	44	46
10–50	71	86	42	30	90	90	45	47	81	91	52	53	68	87	50	47	29	79	25	20	88	85	14	35	88	88	29	55	66	87	41	36	93	83	4	17	96	89	42	39
10–100	75	87	44	29	85	87	30	38	87	89	45	41	68	87	33	37	37	81	40	29	82	82	6	26	76	81	12	30	64	83	31	29	93	81	0	12	97	88	32	35
NEGAR^k	72	86	39	18	63	89	36	34	74	84	30	16	60	85	16	23	60	89	46	38	69	89	27	35	53	88	22	30	42	88	28	18	64	85	0	-7	67	87	7	28
GCN^k	73	87	32	24	90	93	57	58	89	87	32	30	64	86	19	30	69	92	52	61	89	92	64	62	88	96	78	76	66	92	56	44	83	87	17	22	86	90	33	48
GAT^k	80	88	42	27	91	93	64	58	91	87	33	30	69	86	30	28	74	93	64	61	89	92	64	62	90	97	78	84	66	92	56	42	91	90	33	34	94	90	47	51

For ArchStudio, Hadoop, Chromium, and especially OODT, the GT decompositions contain many modules; for example, OODT has $k_{\mathrm{GT}} > 200$, whereas smaller ranges, such as 10–20 and 10–30, enforce a much coarser partition. In these systems, expanding $\mathcal{K}$ generally increases MoJoFM as the recovered architecture becomes more comparable to the GT. In OODT, as $\mathcal{K}$ widens, all metrics increase, indicating that the smaller ranges are too coarse for the reference granularity; the exact-k setting further confirms this. This also shows that the learned representations can support finer-grained decompositions when the target granularity requires it. ArchStudio and Chromium follow the same direction in MoJoFM, and Hadoop shows a similar trend in MoJoFM, while ARI and $\mathrm{C2C}_{\mathrm{cvg}}$ can change more noticeably when $\mathcal{K}$ becomes very wide. Since $\mathrm{C2C}_{\mathrm{cvg}}$ is coverage threshold-based, we interpret it together with ARI.

In contrast, systems with comparatively small GT module counts are more sensitive to over-fragmentation. JabRef illustrates this clearly; as $\mathcal{K}$ widens, MoJoFM remains high, but $\mathrm{C2C}_{\mathrm{cvg}}$ drops from 14 to 0 and ARI drops from 24 to 12, indicating that the recovered decomposition becomes too fine-grained relative to the reference modules. Under exact-k, both metrics increase again (ARI 34, $\mathrm{C2C}_{\mathrm{cvg}}$ 33), showing that the embeddings can also support a coarse decomposition when the target granularity is known.

Bash, HDC, and HDF show the same fragmentation trend as $\mathcal{K}$ widens. ARI and $\mathrm{C2C}_{\mathrm{cvg}}$ decrease while MoJoFM remains comparatively high. For example, on HDF, ARI drops from 79 to 30, and $\mathrm{C2C}_{\mathrm{cvg}}$ drops from 57 to 12. The exact-k results in the second block of Table 5 further show that setting $k = k_{\mathrm{GT}}$ can improve GT alignment on several systems, especially when k_{GT} is small or far from the 10–30 range.

To complement the GT alignment results in Table 2 with an internal modularization view, Table 6 reports the normalized TurboMQ scores, number of recovered clusters, and includes the GT scores as a reference. The table shows that higher internal modularity does not necessarily imply better GT alignment, because TurboMQ measures cohesion and separation within the recovered decomposition, whereas GT metrics assess similarity to the reference architecture. For example, NEGAR often achieves high normalized TurboMQ, consistent with its structure-only design based on second-order random walks. SADE and SARIF can also achieve high scores when they produce relatively compact decompositions. For this reason, we interpret normalized TurboMQ alongside the cluster count and the GT-alignment metrics.

Table 6. Normalized TurboMQ (T) and recovered cluster count $\tilde{k}$. The first row reports the GT decomposition. For GAER and NEGAR, results are medians of T and $\tilde{k}$ over 100 runs with $\mathcal{K} = [10, 30]$.

	AS4		Bash		Chromium		Hadoop		OODT		HDC		HDF		Libxml		JabRef		T.Mates	
Tech.	T	$\tilde{k}$	T	$\tilde{k}$	T	$\tilde{k}$	T	$\tilde{k}$	T	$\tilde{k}$	T	$\tilde{k}$	T	$\tilde{k}$	T	$\tilde{k}$	T	$\tilde{k}$	T	$\tilde{k}$
GT	25	57	24	14	46	67	19	67	79	216	38	11	31	9	31	9	36	6	11	15
ACDC	14	100	28	77	26	784	13	83	36	133	60	17	6	51	36	8	—	—	—	—
FCA	7	160	87	21	29	2 360	9	137	26	222	43	63	5	62	24	21	—	—	—	—
SADE	7	26	67	6	82	34	40	9	92	23	70	12	34	9	30	6	—	—	—	—
SARIF	62	25	16	68	79	22	55	16	72	29	64	11	67	15	41	11	45	29	25	15
NEGAR	87	13	65	18	79	20	64	17	88	17	70	13	48	16	22	15	61	18	31	20
GCN	47	18	33	17	61	18	41	19	69	17	59	18	28	16	33	19	38	17	5	18
GAT	58	18	46	16	72	18	52	18	76	17	60	18	39	17	37	19	48	17	11	18

Answer to RQ2: Varying the number of clusters strongly affects the recovered decomposition and the resulting GT-alignment scores. Smaller k values typically yield higher-level structures that are easier to inspect, whereas larger k values expose more detail and can improve MoJoFM when the reference decomposition is itself detailed. However, increasing k also increases the risk of splitting reference modules, which often lowers ARI and C2C_{cvg}. Overall, the number of clusters should be treated as a modeling choice based on the desired level of architectural detail, since it directly affects both the recovered decomposition and its agreement with the reference architecture.

5.3 Stability of Stochastic Architecture Recovery Methods (RQ3)

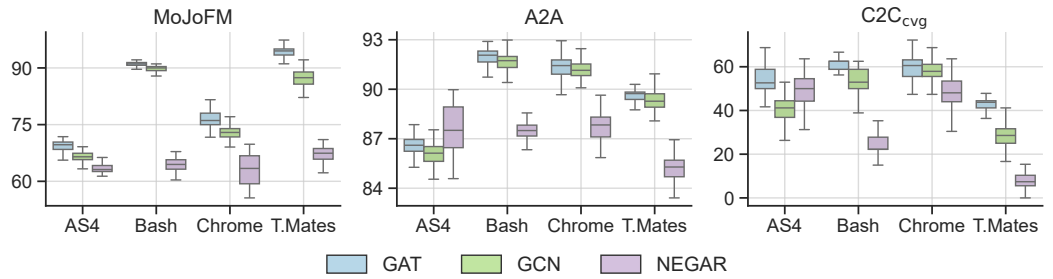
RQ3 evaluates the run-to-run stability of the stochastic methods in our study, GAER (GAT/GCN) and NEGAR, across different random seeds. For each dataset, we run each method 100 times in the range search setting $\mathcal{K} = (10, 30)$ and record the resulting metrics. We compare variability using variance ratios, $\text{VR} = \text{Var}(A)/\text{Var}(B)$, and test for significant differences in variance using the Brown-Forsythe test [3], which compares the average absolute deviation from the median. Table 7 reports pairwise variance ratios for GAT vs. GCN and GAT vs. NEGAR.

Table 7. Run-to-run variability over 100 runs using variance ratios (VR). We report $\text{VR} = \text{Var}(A)/\text{Var}(B)$ for two method pairs, where $\text{VR} > 1$ means A is more variable than B and $\text{VR} < 1$ indicates the opposite. Entries marked with † are not statistically significant ($p \geq 0.05$). Highlighted cells (VR> 1) indicate cases where the second method is significantly less variable than the first.

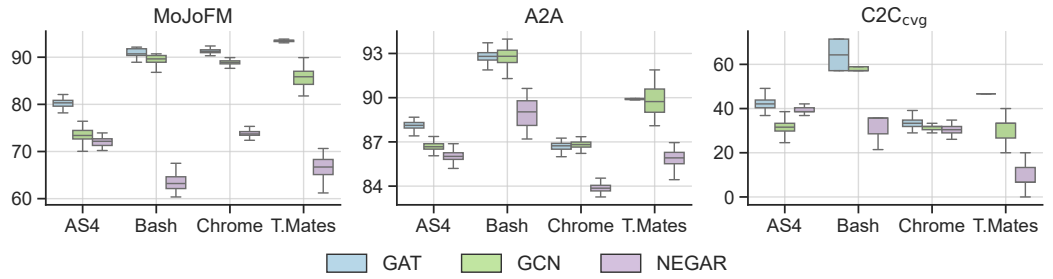
	GAT / GCN				GAT / NEGAR			
Dataset	MoJoFM	A2A	C2C_{cvg}	ARI	MoJoFM	A2A	C2C_{cvg}	ARI
AS4	$0.78^{\dagger}$	$0.70^{\dagger}$	$0.87^{\dagger}$	$1.31^{\dagger}$	$1.41^{\dagger}$	**0.17**	$1.01^{\dagger}$	**0.20**
Bash	$0.46^{\dagger}$	**0.09**	**0.31**	**0.05**	**0.09**	**0.03**	**0.14**	**0.05**
Chromium	$0.91^{\dagger}$	**0.61**	$0.73^{\dagger}$	$0.76^{\dagger}$	**0.68**	**0.69**	$0.75^{\dagger}$	$1.01^{\dagger}$
Hadoop	$0.82^{\dagger}$	$0.80^{\dagger}$	$1.05^{\dagger}$	**1.45**	$0.79^{\dagger}$	$0.95^{\dagger}$	**0.58**	**2.49**
OODT	**1.66**	**1.61**	$1.27^{\dagger}$	**1.90**	**0.37**	**0.35**	**0.24**	**0.41**
HDC	**0.58**	**0.71**	**0.75**	**0.78**	**0.12**	**0.48**	$1.63^{\dagger}$	$0.99^{\dagger}$
HDF	$0.67^{\dagger}$	$1.18^{\dagger}$	**2.09**	$1.11^{\dagger}$	**0.33**	$1.05^{\dagger}$	$0.97^{\dagger}$	**4.56**
Libxml	**2.15**	**1.73**	**2.87**	**3.20**	**1.39**	$1.18^{\dagger}$	$0.68^{\dagger}$	$1.06^{\dagger}$
JabRef	**2.47**	**0.62**	$0.88^{\dagger}$	**0.60**	$1.52^{\dagger}$	**0.66**	$0.98^{\dagger}$	**2.65**
T.Mates	**0.15**	$0.59^{\dagger}$	**0.40**	**1.41**	**0.14**	**0.58**	$1.16^{\dagger}$	$1.00^{\dagger}$

Overall, GAT is as stable as, or more stable than, GCN across many datasets and metrics. The clearest reductions in variability are observed for Bash and HDC, where GAT exhibits significantly lower variability across several metrics. There are also dataset- and metric-specific exceptions in which GCN is significantly less variable, most notably on Libxml (across metrics) and in a few isolated cases, such as JabRef (MoJoFM) and Hadoop/T.Mates (ARI). Compared to NEGAR, GAT often shows lower variability on MoJoFM and A2A (e.g., Bash, Chromium, and T.Mates), while differences on $C2C_{cvg}$ and ARI are more mixed. Lower variability does not necessarily imply better alignment with the GT architecture; a method can be stable across runs while still producing a consistently weaker decomposition.

Figure 2 shows box plots across 100 runs for three metrics (MoJoFM, A2A, and $C2C_{cvg}$) on four representative systems. The plots show that MoJoFM is comparatively stable across methods, whereas the largest spread is typically observed for $C2C_{cvg}$, which is sensitive to fragmentation and to small changes in the recovered partition. The bottom row shows that fixing granularity to the GT cluster count makes the distributions noticeably tighter across metrics and methods, indicating that a large part of the observed variability arises from selecting k within the search range rather than from stochastic training alone. Across the shown systems, GAT is typically the most consistent, with tighter spreads than GCN and NEGAR, especially on A2A and $C2C_{cvg}$. In practice, when k is unknown, running the pipeline a few times (e.g., 2–3) and selecting the solution corresponding to the median-selected k yields a stable estimate of the decomposition.



(a) Cluster range search ($k \in [10, 30]$).



(b) Fixed Clusters ($k = k_{GT}$).

Fig. 2. Run-to-run variability over 100 seeds on four representative systems. Top: k selected via range search. Bottom: k fixed to the reference module count.

Answer to RQ3: GAER shows strong run-to-run stability across datasets. GAER-GAT is often as stable as or more stable than GAER-GCN, with some dataset- and metric-specific exceptions. Compared to NEGAR, both GAER variants are typically less variable on MoJoFM and A2A. Variability is highest for the fragmentation-sensitive metric $C2C_{cvg}$, especially when k is selected from a search range, because small changes in granularity can split or merge reference modules. Fixing the number of clusters reduces variability for all methods, since allowing k to vary across runs introduces additional variation in the recovered partitions and the resulting metrics.

6 Related Work

Architecture recovery aims to infer higher-level architectural modules from implementation artifacts. Prior work differs mainly in the information sources it exploits (dependencies, code semantics, directory structure) and in how these signals are combined to derive a decomposition (search-based optimization, clustering, heuristic fusion, representation learning, or LLM-guided mapping). Table 8 summarizes representative techniques by information source and recovery strategy.

Traditional SAR models treat software artifacts as a dependency graph and recover modules using several related paradigms. A first group comprises search-based methods that optimize modularization objectives. Within this group, some methods optimize a single modularization quality (MQ) objective, favoring high cohesion within modules and low coupling across modules [25, 27, 30, 39], while others use multi-objective or evolutionary formulations to jointly optimize multiple criteria [15]. A second group comprises clustering-based methods, which merge artifacts using dependency-derived similarity or information-theoretic criteria [1, 34, 35]. In addition, some structural methods emphasize practicality, determinism, or scalability in recovery [32, 40, 48]. In practice, these approaches can be effective when dependencies reflect architectural structure, but recovery quality remains sensitive to how the dependency graph is constructed and how dependency relations are represented or weighted.

Beyond structural approaches, semantic approaches use identifiers and comments to cluster artifacts and may outperform dependency-only methods in certain scenarios [10]. Representative techniques cluster artifacts in a latent semantic space learned from source code (e.g., topic models or LSI) or directly from lexical similarity features [8, 13, 43]. Kargar et al. [19] construct a semantic dependency graph from lexical cues and optimize the decomposition via local search. Furthermore, Anquetil et al. [2] proposed a directory-based recovery approach that uses file names and directory paths as proxies for the architecture. However, these non-structural signals can be noisy in isolation because widely reused terminology and utility directories mix entities from many modules.

Hybrid approaches combine multiple information sources to improve robustness. Prior work includes weighted similarity fusion with search-based partitioning and many-objective remodularization, which augment structural objectives with semantic and evolution-related criteria [29, 31, 42, 45]. Puchala et al. [41] combine multiple clustering results into a consensus solution, while Yang et al. [55] used semantic cues to guide propagation over the dependency graph. Papachristou et al. [38] and Zhang et al. [56] fuse signals by constructing weighted dependency graphs and applying community detection. Overall, most hybrid techniques rely on fixed fusion rules, heuristic weighting, or late-stage aggregation, rather than learning a unified representation that can jointly model structural and semantic signals.

Representation learning has also been explored for SAR. Chen et al. [4] proposed NEGAR, which learns embeddings from static dependency graphs via random walks and clusters the resulting embeddings to find architectural decompositions. They report that recovery quality varies across dependency types, motivating methods that can jointly exploit multiple relations. However, NEGAR runs a separate pipeline for each dependency type and does not incorporate non-structural signals.

Table 8. Overview of SAR techniques grouped by information source and recovery strategy. Dep: static dependencies; Sem: textual features from code/comments; Dir: directory structure/package names.

Technique	Dep.	Sem.	Dir.	Recovery Strategy/Clustering Method
1) Dependency Graph-Based Approaches				
Bunch–NAHC/SAHC [30]	✓	✗	✗	Local-search based graph partitioning; hill-climbing variants; optimize MQ.
WCA [27]	✓	✗	✗	Weighted hierarchical clustering; feature-vector similarity; optimize MQ.
MCA & ECA [15]	✓	✗	✗	Multi-objective and evolutionary clustering (genetic algorithm); optimize MQ.
DAGC [39]	✓	✗	✗	Global search partitioning; genetic algorithm+permutation encoding; optimize MQ.
ACDC [49]	✓	✗	✗	Pattern-driven clustering: builds a subsystem skeleton and uses orphan adoption.
CCT [35]	✓	✗	✗	Cooperative hierarchical clustering using multiple similarity measures.
LIMBO [1]	✓	✗	✗	Information-bottleneck hierarchical clustering (information-theoretic merging).
FCA [48]	✓	✗	✗	Deterministic fast clustering on neighborhood degree matrix; designed for scalability.
Mohammadi et al. [32]	✓	✗	✗	Graph-guided heuristic clustering based on neighborhood structure.
HCC [34]	✓	✗	✗	Ensemble hierarchical clustering; combine multiple dendrograms into consensus.
GMA [40]	✓	✗	✗	Graph-similarity clustering with k-medoids (iterative refinement).
2) Directory-Based Approaches				
Anquetil et al. [2]	✗	✗	✓	Cluster using file names and directory paths (substring/n-gram similarity).
3) Semantic-Based Approaches				
ARC [13]	✗	✓	✗	LDA extracts topics ("concerns") from code/comments; cluster by topic distributions.
ZBR [8]	✗	✓	✗	TF-IDF lexical zones with weights learned via expectation maximization → hierarchical clustering.
Risi et al. [43]	✗	✓	✗	Cluster in LSI latent space with k-means; fold-in/fold-out updates without recomputing the model.
Kargar et al. [19]	✗	✓	✗	Build a semantic dependency graph from lexical cues → hill-climbing search to optimize MQ.
4) Hybrid / Multi-source Approaches				
Misra et al. [29]	✓	✓	✗	Automated feature weighting → weighted graph partitioning (multi-objective modularity criterion).
NSGA-III [31]	✓	✓	✗	Multi-objective optimization, semantics, and change-history consistency.
EoD/CGH/CGoH [45]	✓	✓	✗	Optimize a multi-objective fitness function combining multiple signals with various search strategies.
ESCRC [42]	✓	✓	✗	Combines structural coupling & conceptual text similarity and applies hierarchical clustering.
Puhala et al. [41]	✓	✓	✓	Generate diverse base clusterings across Dep/Sem/Dir → evidence-accumulation consensus clustering.
EVOL [55]	✓	✓	✗	Filter semantic outliers → build clustering skeleton → label propagation on dependency graph.
SADE [38]	✓	✓	✗	Semantic weighting → weighted call graph → community detection.
SARIF [56]	✓	✓	✓	Heuristic information fusion → weighted graph → Louvain community detection.
5) Representation Learning				
NEGAR [4]	✓	✗	✗	Random-walk graph embeddings (node2vec/SkipGram) → k-means clustering.
6) LLM-Based Approaches				
Rukmono et al. [44]	✗	✓	✗	GPT-4-guided deductive mapping to the reference architecture; chains-of-thought prompting.

Complementary to bottom-up approaches, Rukmono et al. [44] use chains-of-thought prompting on GPT-4 to map implementation units to a predefined reference architecture. However, this approach requires an available architectural model and raises scalability concerns.

7 Threats to Validity

External Validity. Our evaluation relies on ten open-source systems in C, C++, and Java, each with a published ground-truth C2A mapping. While these systems provide a solid benchmark for comparing approaches, ten projects across three languages represent only a small sample relative to the diversity of real-world software and do not cover all architectural styles or sizes.

Internal Validity. We made a strong effort to match the system versions used in prior studies. However, some benchmarks are more than 10 years old, so an exact match is not always verifiable (e.g., Chromium). Some entities in the ground truth are generated during the build process, so the build environment must also be configured consistently. We reproduced values comparable to those reported in the SARIF and NEGAR study, suggesting that any remaining dataset differences are unlikely to affect our conclusions.

The dependencies were extracted using a static analysis tool, which may miss some interactions due to language limitations, dynamic features, or incomplete parsing. Although this affects all dependency-based recovery techniques, it can lead to an under-representation of actual cohesion and coupling in the input graph. We decided to drop one system used in previous studies, ITK, because we could not reliably extract dependencies from it, and we could not reproduce prior baseline results on that system.

The reliability of the ground-truth architectures themselves is another concern. Humans create these labels, which may be subject to interpretation, since system experts do not always agree on every assignment. For some systems (e.g., JabRef, TeamMates, HDC, HDF, and Libxml), the mappings were provided directly by system experts or industry partners. For others (e.g., Archstudio, Bash, Chromium, Hadoop, and OODT), researchers derived them from documentation and implementation details. Although prior studies have shown such reconstructions to be consistent and stable over time, the possibility of errors or biases remains, which may influence the evaluation of recovery techniques.

A further threat concerns the extent to which benchmark-based evaluation reflects real-world architecture recovery practice. We compare GAER with strong recent baselines, including SARIF and NEGAR, using standard recovery metrics and identical experimental conditions across datasets. To reduce reliance on a single benchmark suite and a single style of ground-truth construction, we also include systems with expert-provided architectures (JabRef and TeamMates; Section 4.1). This setup supports a fair and reproducible comparison, although conclusions may still depend on the characteristics of the available benchmarks.

From a statistical perspective, with 100 runs per method, tests can detect even small systematic differences, so very low p-values may occur even when practical differences are minor. This can occur, for example, for A2A when scores are uniformly high across methods, yet small non-random gaps remain. We therefore emphasize effect sizes alongside p-values and report Cliff's delta to indicate whether differences are practically meaningful rather than only statistically detectable.

8 Conclusion

We presented GAER, a graph auto-encoder framework for unsupervised recovery of software architectures. We studied two encoder variants, GAER-GAT and GAER-GCN. Our approach models software systems as heterogeneous, multi-relational dependency graphs and integrates structural dependencies, directory hierarchy, and code semantics as node features. By training the encoder to reconstruct observed dependencies, it learns architecture-aware embeddings without requiring handcrafted fusion heuristics or labeled data. Architectural boundaries are then recovered by clustering the learned node representations.

Across the evaluated systems, GAER yields decompositions that align well with the reference architectures and are competitive with strong established baselines. GAT is often the stronger encoder, with GCN typically close behind. On average, GAER-GAT improves over SARIF by about +14 MoJoFM, +3 A2A, +19 $C2C_{cvg}$, and +14 ARI, and over NEGAR by about +20 MoJoFM, +3 A2A, +23 $C2C_{cvg}$, and +18 ARI (Table 3).

We also found that the number of clusters used to form the final decomposition substantially influences the recovered architecture and the reported metrics. Smaller cluster counts typically yield

higher-level structures that are easier to inspect, whereas larger cluster counts reveal more detail and can improve MoJoFM when the reference decomposition is itself fine-grained. At the same time, larger cluster counts increase the risk of splitting reference modules, thereby reducing ARI and $C2C_{cvg}$. Finally, repeated-run analysis indicates that GAER is generally stable across datasets.

Overall, GAER demonstrates the potential of graph representation learning as a scalable, data-driven alternative to manual and heuristic-based recovery techniques. By reducing the effort needed to maintain architectural documentation, GAER supports a more effective understanding and evolution of complex software systems.

Future research could focus on integrating the clustering objective directly into the GAE training process to jointly learn embeddings and architectural boundaries. Another promising direction is to investigate sparsification strategies to handle noisy or redundant dependencies in dense graphs, thereby improving performance and efficiency. Finally, extending the framework to semi-supervised settings, in which limited expert input guides recovery, could improve interpretability and alignment with the intended design. A natural extension is to explore automatic naming or interpretation of recovered clusters using language models or code summarization techniques to enhance comprehensibility.

References

[1] Periklis Andritsos, Panayiotis Tsaparas, Renée J Miller, and Kenneth C Sevcik. 2004. LIMBO: Scalable clustering of categorical data. In *International Conference on Extending Database Technology*. Springer, 123–146. https://doi.org/10.1007/978-3-540-24741-8_9

[2] Nicolas Anquetil and Timothy C Lethbridge. 1999. Recovering software architecture from the names of source files. *Journal of Software Maintenance: Research and Practice* 11, 3 (1999), 201–221. https://doi.org/10.1002/(sici)1096-908x(199905/06)11:3<201::aid-smr192>3.0.co;2-1

[3] Morton B. Brown and Alan B. Forsythe. 1974. Robust Tests for the Equality of Variances. *J. Amer. Statist. Assoc.* 69, 346 (1974), 364–367. https://doi.org/10.1080/01621459.1974.10482955

[4] Jiayi Chen, Zhixing Wang, Yuchen Jiang, Jun Pang, Tian Zhang, Minxue Pan, and Jianwen Sun. 2022. NEGAR: network embedding guided architecture recovery for software systems. In *2022 29th Asia-Pacific Software Engineering Conference (APSEC)*. IEEE, IEEE, 367–376. https://doi.org/10.1109/apsec57359.2022.00049

[5] Aaron Clauset, Mark EJ Newman, and Cristopher Moore. 2004. Finding community structure in very large networks. *Physical Review E—Statistical, Nonlinear, and Soft Matter Physics* 70, 6 (2004), 066111. https://doi.org/10.1103/physreve.70.066111

[6] Paul Clements, David Garlan, Reed Little, Robert Nord, and Judith Stafford. 2003. Documenting software architectures: views and beyond. In *25th International Conference on Software Engineering, 2003. Proceedings.* IEEE, 740–741. https://doi.org/10.1109/icse.2003.1201264

[7] Norman Cliff. 1993. Dominance statistics: Ordinal analyses to answer ordinal questions. *Psychological bulletin* 114, 3 (1993), 494. https://doi.org/10.1037/0033-2909.114.3.494

[8] Anna Corazza, Sergio Di Martino, Valerio Maggio, and Giuseppe Scanniello. 2011. Investigating the use of lexical information for software system clustering. In *2011 15th European Conference on Software Maintenance and Reengineering*. IEEE, 35–44. https://doi.org/10.1109/csmr.2011.8

[9] Lakshitha De Silva and Dharini Balasubramaniam. 2012. Controlling software architecture erosion: A survey. *Journal of Systems and Software* 85, 1 (2012), 132–151. https://doi.org/10.1016/j.jss.2011.07.036

[10] Joshua Garcia, Igor Ivkovic, and Nenad Medvidovic. 2013. A comparative analysis of software architecture recovery techniques. In *2013 28th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 486–496. https://doi.org/10.1109/ase.2013.6693106

[11] Joshua Garcia, Ivo Krka, Chris Mattmann, and Nenad Medvidovic. 2013. Obtaining ground-truth software architectures. In *2013 35th International Conference on Software Engineering (ICSE)*. IEEE, 901–910. https://doi.org/10.1109/icse.2013.6606639

[12] Joshua Garcia, Duc Le, Daniel Link, AS Pooyan Behnamghader, Eder Figueroa Ortiz, and Nenad Medvidovic. 2014. An empirical study of architectural change and decay in open-source software systems. *USC-CSSE, Tech. Rep* (2014).

[13] Joshua Garcia, Daniel Popescu, Chris Mattmann, Nenad Medvidovic, and Yuanfang Cai. 2011. Enhancing architectural recovery using concerns. In *2011 26th IEEE/ACM International Conference on Automated Software Engineering (ASE 2011)*. IEEE, 552–555. https://doi.org/10.1109/ase.2011.6100123

[14] Aditya Grover and Jure Leskovec. 2016. node2vec: Scalable feature learning for networks. In *Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining*. 855–864. https://doi.org/10.1145/2939672.2939754

[15] Mark Harman and Xin Yao. 2010. Software module clustering as a multi-objective search problem. *IEEE Transactions on Software Engineering* 37, 2 (2010), 264–282. https://doi.org/10.1109/tse.2010.26

[16] Lawrence Hubert and Phipps Arabie. 1985. Comparing partitions. *Journal of classification* 2, 1 (1985), 193–218. https://doi.org/10.1007/bf01908075

[17] Rakhshanda Jabeen, Morgan Ericsson, Jonas Nordqvist, and Anna Wingkvist. 2025. Graph Convolution Networks for Mapping Source Code Entities to Architectural Modules. In *2025 IEEE 22nd International Conference on Software Architecture Companion (ICSA-C)*. IEEE, 472–481. https://doi.org/10.1109/icsa-c65153.2025.00071

[18] Rakhshanda Jabeen, Morgan Ericsson, Jonas Nordqvist, and Anna Wingkvist. 2026. *Reproduction package for GAER: Graph Auto-Encoders for Unsupervised Software Architecture Recovery*. https://doi.org/10.5281/zenodo.19534384

[19] Masoud Kargar, Ayaz Isazadeh, and Habib Izadkhah. 2017. Semantic-based software clustering using hill climbing. In *2017 international symposium on computer science and software engineering conference (CSSE)*. IEEE, 55–60. https://doi.org/10.1109/csicsse.2017.8320117

[20] Thomas N Kipf and Max Welling. 2016. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907* (2016). https://doi.org/10.48550/arXiv.1609.02907

[21] Thomas N Kipf and Max Welling. 2016. Variational graph auto-encoders. *arXiv preprint arXiv:1611.07308* (2016). https://doi.org/10.48550/arXiv.1611.07308

[22] Duc Minh Le, Pooyan Behnamghader, Joshua Garcia, Daniel Link, Arman Shahbazian, and Nenad Medvidovic. 2015. An empirical study of architectural change in open-source software systems. In *2015 IEEE/ACM 12th working conference on mining software repositories*. IEEE, 235–245. https://doi.org/10.1109/msr.2015.29

[23] Thibaud Lutellier, Devin Chollak, Joshua Garcia, Lin Tan, Derek Rayside, Nenad Medvidovic, and Robert Kroeger. 2015. Comparing software architecture recovery techniques using accurate dependencies. In *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*, Vol. 2. IEEE, 69–78. https://doi.org/10.1109/icse.2015.136

[24] Thibaud Lutellier, Devin Chollak, Joshua Garcia, Lin Tan, Derek Rayside, Nenad Medvidović, and Robert Kroeger. 2017. Measuring the impact of code dependencies on software architecture recovery techniques. *IEEE Transactions on Software Engineering* 44, 2 (2017), 159–181. https://doi.org/10.1109/tse.2017.2671865

[25] Spiros Mancoridis, Brian S Mitchell, Yihfarn Chen, and Emden R Gansner. 1999. Bunch: A clustering tool for the recovery and maintenance of software system structures. In *Proceedings IEEE International Conference on Software Maintenance-1999 (ICSM'99).'Software Maintenance for Business Change'(Cat. No. 99CB36360)*. IEEE, 50–59. https://doi.org/10.1109/icsm.1999.792498

[26] Henry B Mann and Donald R Whitney. 1947. On a test of whether one of two random variables is stochastically larger than the other. *The annals of mathematical statistics* (1947), 50–60. https://doi.org/10.1214/aoms/1177730491

[27] Onaiza Maqbool and Haroon Atique Babri. 2004. The weighted combined algorithm: A linkage algorithm for software clustering. In *Eighth European Conference on Software Maintenance and Reengineering, 2004. CSMR 2004. Proceedings.* IEEE, 15–24. https://doi.org/10.1109/csmr.2004.1281402

[28] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S Corrado, and Jeff Dean. 2013. Distributed representations of words and phrases and their compositionality. *Advances in neural information processing systems* 26 (2013).

[29] Janardan Misra, KM Annervaz, Vikrant Kaulgud, Shubhashis Sengupta, and Gary Titus. 2012. Software clustering: Unifying syntactic and semantic features. In *2012 19th Working Conference on Reverse Engineering*. IEEE, 113–122. https://doi.org/10.1109/wcre.2012.21

[30] Brian S Mitchell and Spiros Mancoridis. 2006. On the automatic modularization of software systems using the bunch tool. *IEEE Transactions on Software Engineering* 32, 3 (2006), 193–208. https://doi.org/10.1109/tse.2006.31

[31] Wiem Mkaouer, Marouane Kessentini, Adnan Shaout, Patrice Koligheu, Slim Bechikh, Kalyanmoy Deb, and Ali Ouni. 2015. Many-objective software remodularization using NSGA-III. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 24, 3 (2015), 1–45. https://doi.org/10.1145/2729974

[32] Sina Mohammadi and Habib Izadkhah. 2019. A new algorithm for software clustering considering the knowledge of dependency between artifacts in the source code. *Information and Software Technology* 105 (2019), 252–256. https://doi.org/10.1016/j.infsof.2018.09.001

[33] Fionn Murtagh and Pierre Legendre. 2014. Ward's hierarchical agglomerative clustering method: which algorithms implement Ward's criterion? *Journal of classification* 31 (2014), 274–295. https://doi.org/10.1007/s00357-014-9161-z

[34] Rashid Naseem, Mustafa Mat Deris, Onaiza Maqbool, and Sara Shahzad. 2019. Euclidean space based hierarchical clusterers combinations: an application to software clustering. *Cluster Computing* 22, Suppl 3 (2019), 7287–7311. https://doi.org/10.1007/s10586-017-1408-0

[35] Rashid Naseem, Onaiza Maqbool, and Siraj Muhammad. 2013. Cooperative clustering for software modularization. *Journal of Systems and Software* 86, 8 (2013), 2045–2062. https://doi.org/10.1016/j.jss.2013.03.080

[36] Hung Viet Nguyen. 2023. Depends: Multilingual Dependency Analysis Tool. https://github.com/multilang-depends/depends. Accessed: 2025-08-17.
[37] Tobias Olsson, Morgan Ericsson, and Anna Wingkvist. 2022. To automatically map source code entities to architectural modules with Naive Bayes. *Journal of Systems and Software* 183 (2022), 111095. https://doi.org/10.1016/j.jss.2021.111095
[38] Marios Papachristou. 2019. Software clusterings with vector semantics and the call graph. In *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1184–1186. https://doi.org/10.1145/3338906.3342483
[39] Saeed Parsa and Omid Bushehrian. 2005. A new encoding scheme and a framework to investigate genetic clustering algorithms. *Journal of Research and Practice in Information Technology* 37, 1 (2005), 127–143.
[40] Babak Pourasghar, Habib Izadkhah, Ayaz Isazadeh, and Shahriar Lotfi. 2021. A graph-based clustering algorithm for software systems modularization. *Information and Software Technology* 133 (2021), 106469. https://doi.org/10.1016/j.infsof.2020.106469
[41] Shiva Prasad Reddy Puchala, Jitender Kumar Chhabra, and Amit Rathee. 2022. Ensemble clustering based approach for software architecture recovery. *International Journal of Information Technology* 14, 4 (2022), 2013–2019. https://doi.org/10.1007/s41870-021-00846-0
[42] Amit Rathee and Jitender Kumar Chhabra. 2017. Software remodularization by estimating structural and conceptual relations among classes and using hierarchical clustering. In *Advanced Informatics for Computing Research: First International Conference, ICAICR 2017, Jalandhar, India, March 17–18, 2017, Revised Selected Papers*. Springer, 94–106. https://doi.org/10.1007/978-981-10-5780-9_9
[43] Michele Risi, Giuseppe Scanniello, and Genoveffa Tortora. 2012. Using fold-in and fold-out in the architecture recovery of software systems. *Formal Aspects of Computing* 24, 3 (2012), 307–330. https://doi.org/10.1007/s00165-011-0199-y
[44] Satrio Adi Rukmono, Lina Ochoa, and Michel Chaudron. 2024. Deductive software architecture recovery via chain-of-thought prompting. In *Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results*. 92–96. https://doi.org/10.1145/3639476.3639776
[45] Nafiseh Sadat Jalali, Habib Izadkhah, and Shahriar Lotfi. 2019. Multi-objective search-based software modularization: Structural and non-structural features. *Soft Computing-A Fusion of Foundations, Methodologies & Applications* 23, 21 (2019). https://doi.org/10.1007/s00500-018-3666-z
[46] Anthony Savidis and Crystalia Savaki. 2022. Software Architecture Mining from Source Code with Dependency Graph Clustering and Visualization.. In *VISIGRAPP (3: IVAPP)*. 179–186. https://doi.org/10.5220/0010896800003124
[47] Ioanna Stavropoulou, Marios Grigoriou, and Kostas Kontogiannis. 2017. Case study on which relations to use for clustering-based software architecture recovery. *Empirical Software Engineering* 22, 4 (2017), 1717–1762. https://doi.org/10.1007/s10664-016-9459-z
[48] Navid Teymourian, Habib Izadkhah, and Ayaz Isazadeh. 2020. A fast clustering algorithm for modularization of large-scale software systems. *IEEE Transactions on Software Engineering* 48, 4 (2020), 1451–1462. https://doi.org/10.1109/tse.2020.3022212
[49] Vassilios Tzerpos and Richard C Holt. 2000. ACDC: an algorithm for comprehension-driven clustering. In *Proceedings Seventh Working Conference on Reverse Engineering*. IEEE, 258–267. https://doi.org/10.1109/wcre.2000.891477
[50] Universal Ctags Team. 2024. *Universal Ctags*. https://github.com/universal-ctags/ctags Accessed: 2026-01-25.
[51] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
[52] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. 2017. Graph attention networks. *arXiv preprint arXiv:1710.10903* (2017). https://doi.org/10.48550/arXiv.1710.10903
[53] Zhihua Wen and Vassilios Tzerpos. 2004. An effectiveness measure for software clustering algorithms. In *Proceedings. 12th IEEE International Workshop on Program Comprehension, 2004.* IEEE, 194–203. https://doi.org/10.1109/wpc.2004.1311061
[54] Frank Wilcoxon. 1945. Individual comparisons by ranking methods. *Biometrics bulletin* 1, 6 (1945), 80–83. https://doi.org/10.2307/3001968
[55] Kaiyuan Yang, Junfeng Wang, Zhiyang Fang, Peng Wu, and Zihua Song. 2022. Enhancing software modularization via semantic outliers filtration and label propagation. *Information and Software Technology* 145 (2022), 106818. https://doi.org/10.1016/j.infsof.2021.106818
[56] Yiran Zhang, Zhengzi Xu, Chengwei Liu, Hongxu Chen, Jianwen Sun, Dong Qiu, and Yang Liu. 2023. Software architecture recovery with information fusion. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1535–1547. https://doi.org/10.1145/3611643.3616285

Received 2026-02-25; accepted 2026-03-24